DATABASE DECISION | SUPABASE POSTGRES MVP

CropSage Database Storage Design

Final data-retention scope for farm profiles, provider evidence, deterministic scoring, validation and reproducible scenarios.

FINAL SCOPE	**Use 10 PostgreSQL tables. Keep the 22-crop catalog and scoring source code versioned in Git; save the exact versions and hashes used by every recommendation run.**

What The Database Must Guarantee

1. The exact farm profile, evidence, catalog and scoring policy used for a result can be reconstructed.
2. Provider evidence remains traceable to its endpoint, request, time range, spatial resolution and live/cache state.
3. The deterministic engine owns all scores; the validator decides whether those scores may be rendered.
4. Baseline and changed-condition scenarios remain linked without modifying the original farm profile.
5. Missing data remains unknown and lowers confidence; it is never converted into favorable evidence.

End-to-End Persistence Flow

SESSION assessment_sessions	-> **FARM** farm_profiles	-> **PROVIDERS** provider_fetches
EVIDENCE records + bundles	-> **SCORING** recommendation_runs	-> **RESULTS** crop results + validation

Relational columns

- IDs, versions, statuses and timestamps.
- Provider, variable, unit and cache keys.
- Farm point and optional boundary via PostGIS.
- Foreign keys connecting the complete run.

JSONB snapshots

- Optional farm inputs and field-source metadata.
- Provider-specific normalized values.
- Scenario changes and engine input/output.
- Factor details, warnings and validation diagnostics.

CATALOG TRUTH	**The database design follows catalog version 1.1.0 with 22 crops. Older planning references to six crops are superseded.**

TABLE INVENTORY | FOUNDATIONAL AND EVIDENCE DATA

Tables 1-5

The first half of the schema establishes retained session state, immutable farm profiles and traceable provider evidence.

#	Table	Data saved	Why it exists
1	`assessment_sessions`	Session ID; active profile; status; created, updated and expiry timestamps.	Groups baseline and follow-up scenario runs without requiring user accounts.
2	`farm_profiles`	Versioned location, planting time, region, irrigation, optional farmer soil overrides, goals, field sources and missing fields.	Stores trusted farm state. Existing versions are never overwritten.
3	`provider_fetches`	Provider, endpoint, sanitized parameters, cache key, activity ID, request status, timestamps, live/cache/fallback state, errors and raw-file reference.	Tracks API requests, asynchronous FortyGuard jobs, failures and cache reuse.
4	`evidence_records`	Normalized variable, value, unit, source field, time range, point/area, resolution, quality, freshness and warnings.	Stores individual NASA, Open-Meteo, FortyGuard and SSURGO facts.
5	`evidence_bundles`	Farm profile, bundle version/hash, assembly time, provider coverage, freshness, completeness and status.	Represents the complete location evidence supplied to the scorer.

Evidence Request Versus Evidence Fact

provider_fetches: request lifecycle

- Sanitized endpoint and parameters.
- Stable request hash and cache key.
- FortyGuard activity ID and polling status.
- Submitted, completed, fetched and expiry times.
- Live, cached or fallback classification.
- Safe error details and raw-file reference.

evidence_records: normalized facts

- Canonical variable key and normalized value.
- Original provider field and unit.
- Observation and validity time range.
- Point/area and spatial resolution.
- Freshness, quality flags and warnings.
- Provider-fetch link for full provenance.

WHY SEPARATE THEM	**One provider request can produce many normalized facts. Cached facts may be reused without pretending that the provider was called again.**

Tables 6-10

The second half connects approved evidence to deterministic recommendations, validation and user-safe progress events.

#	Table	Data saved	Why it exists
6	`evidence_bundle_records`	Evidence-bundle ID and evidence-record ID, with optional inclusion role/order.	Lets cached evidence be reused across bundles without duplicating provider records.
7	`recommendation_runs`	Baseline/scenario request, immutable engine input/output, catalog and evidence versions, engine/policy versions, status and timing.	Records every deterministic-engine execution and preserves reproducibility.
8	`crop_score_results`	One row per crop: eligibility, rank, suitability, factors, risk, confidence, evidence links and scenario deltas.	Makes deterministic results auditable, comparable and queryable.
9	`validation_reports`	Validator version, errors, warnings, reconciliation, grounding, evidence coverage and render_allowed.	Determines whether a recommendation may be shown to the user.
10	`run_events`	Tool/event name, order, status, timing, cache state, sanitized arguments and user-safe summary.	Supports visible application progress without storing hidden reasoning.

Relationship Map

Parent	Relationship	Child
`assessment_sessions`	has many	`farm_profiles and recommendation_runs`
`provider_fetches`	produces many	`evidence_records`
`evidence_bundles`	includes many through join	`evidence_records`
`recommendation_runs`	evaluates many	`crop_score_results`
`recommendation_runs`	has one final	`validation_reports`
`recommendation_runs`	may reference baseline through	`parent_run_id`

NO EXTRA TABLES	**A separate scenario table, cache table or crop-catalog table is unnecessary for the MVP. Scenario data belongs to recommendation_runs; provider_fetches owns cache metadata; the catalog stays in Git.**

Scoring Engine Persistence Contract

The database stores the exact inputs, versions, per-crop calculations and validated output of the deterministic engine being implemented by the project team.

recommendation_runs: run envelope

- Farm profile ID and exact profile version.
- Evidence bundle ID, schema version and hash.
- Catalog version and catalog-file hash.
- Engine version or Git commit.
- Scoring-policy version and hash.
- Baseline or scenario type and parent run.
- Scenario changes and disclosed assumptions.
- Immutable engine_input JSONB.
- Immutable engine_output JSONB.
- Run status, timing and safe failure details.

crop_score_results: one row per crop

- Crop ID, eligibility and hard-filter reasons.
- Final rank and total suitability score.
- Temperature, climate, soil, water, region and season components.
- Factor contributions and intermediate calculations.
- Heat, soil and water comparison metrics.
- Risk flags, separate from suitability.
- Confidence score/level and confidence inputs.
- Missing and supplementary evidence.
- Evidence-record links and reason codes.
- Score and rank deltas versus baseline.

Validation Boundary

1. The engine calculates eligibility, suitability, factors, risks, confidence inputs and scenario deltas.
2. The validator reconciles totals, verifies crop/catalog grounding, checks evidence freshness and decides render_allowed.
3. The language model may explain validated values but must not calculate, modify or replace them.
4. A saved presentation snapshot is allowed only after validation; it never becomes trusted input for another run.

CORE RULE	**Suitability, risk and confidence remain separate. A strong suitability score cannot erase a risk flag or weak evidence confidence.**

Versioning Needed For Reproduction

Version	Stored on	Why
`farm profile version`	recommendation_runs	Prevents later farm edits from changing a past result.
`evidence bundle hash`	recommendation_runs	Proves the exact location evidence used.
`catalog version + hash`	recommendation_runs	Anchors all crop requirements to catalog 1.1.0 or a later release.
`engine + policy version`	recommendation_runs	Anchors formulas, weights and missing-data behavior.
`validator version`	validation_reports	Explains which release authorized rendering.

What We Save And What We Keep Outside

The final boundary keeps the recommendation reproducible without turning PostgreSQL into a raw-file archive or storing unnecessary farmer information.

Farm Profile Data

Required and typed

- PostGIS farm point: geography(Point, 4326).
- Manual Texas region and assignment method.
- Planting date or planting month.
- Irrigation availability and reliability.
- Profile version, input hash and supersedes_profile_id.
- Created time, source metadata and missing-field list.

Optional and nullable

- Farm boundary polygon.
- Well or canal capacity.
- Farmer soil-texture override.
- Laboratory pH override.
- Current soil moisture or farmer rainfall observation.
- Farmer goal and completeness notes.

SCENARIO RULE	**A scenario stores changed variables in recommendation_runs and links to its baseline through parent_run_id. It does not mutate the original farm profile.**

Keep Outside PostgreSQL Rows

1. The 22-crop catalog itself; keep it as schema-validated JSON in Git.
2. Scoring-engine source code and policy files; keep them versioned in Git.
3. Large heatmaps, rasters and full raw provider payloads.
4. Unsanitized responses, API keys, credentials and temporary signed URLs.
5. Hidden agent reasoning, chain-of-thought and free-text model output as trusted state.
6. Unnecessary names, contact information or other farmer-identifying data.

Raw Evidence File Policy

Sanitized fallback exports may remain in the repository for the hackathon demo or later move to Supabase Storage. PostgreSQL stores only the stable reference, checksum, content type, size, provider timestamp and sanitization status.

Implementation Order

1	Create migrations for sessions, farm profiles and evidence lifecycle tables.
2	Finalize the deterministic engine input/output contract before creating scoring-result columns.
3	Seed the Plainview profile and unified evidence bundle with sanitized references.
4	Persist one baseline run, one scenario run and their validation reports.
5	Add run-event writes when production adapters and the application flow are connected.

READY FOR SCHEMA DESIGN	**This scope covers the full Plainview MVP: retained farm state, provider caching, unified evidence, deterministic scoring, scenarios, validation, reproducibility and later UI progress.**

Internal basis: ARCHITECTURE.md, FARM_PROFILE_INPUTS.pdf, catalog 1.1.0, catalog evidence policy and the current Plainview evidence workflow.